With any of these forms, you can pass in another RSpec matcher (such as a_string_starting_with for the message name) to control how the matching works. For example, the following expectation ensures the exception has its name attribute set:

11-matchers-included-in-rspec-expectations/03/block_matchers.rb
```
expect {
  'hello'.world
}.to raise_error(an_object_having_attributes(name: :world))
```

Checking properties of exceptions can get really fiddly. If you find yourself passing a complex nested composed matcher expression to raise_error, you'll see a really long failure message in the output. To avoid this situation, you can instead pass a block into raise_error and move your logic there:

11-matchers-included-in-rspec-expectations/03/block_matchers.rb
```
expect { 'hello'.world }.to raise_error(NoMethodError) do |ex|
  expect(ex.name).to eq(:world)
end
```

There are a couple of gotchas with raise_error that can lead to false positives. First, raise_error (with no arguments) will match *any* error—and it can't tell the difference between exceptions you *did* or *did not* mean to throw.

For example, if you rename a method but forget to update your spec, Ruby will throw a NoMethodError. An overzealous raise_error will swallow this exception, and your spec will pass even though it's no longer exercising the method you mean to test!

Likewise, you might use raise_error(ArgumentError) to make sure one of your methods correctly raises this error. If you later make a breaking change to the method signature—such as adding an argument—but forget to update a caller, Ruby will throw the same error. Your spec will pass (because all it sees is the ArgumentError it's expecting), but the code will still be broken.

We've actually run into this kind of false positive In RSpec itself.[7]

Never Check for a Bare Exception



Always include some kind of detail—either a specific custom error class or a snippet from the message—that is unique to the specific raise statement you are testing.

On the flip side, the negative form—expect { ... }.not_to raise_error(...)—has the opposite problem. If we give too much detail in our specs, we risk seeing a

7. https://github.com/rspec/rspec-mocks/pull/550